

C: Pointers and Arrays

Jinyang Li

What we've learnt in last lecture

- C program's basic organization
 - *.c vs. *.h files
 - Compilation workflow and the *make* tool

Lesson plan

- Bitwise operators
- Control flow
- Pointers

Bitwise NOT: $\sim$

Truth table (of boolean function NOT)

x	NOT x
0	1
1	0

Operator $\sim$ applies NOT bitwise to two integers

result of $\sim 0x69$

$$\sim (01101001)_2$$

$$(10010110)_2$$

Bitwise AND: &

Truth table (of boolean function AND)

x	y	x AND y
0	0	0
0	1	0
1	0	0
1	1	1

How many rows
if function has
n boolean (aka
1-bit) inputs?

2^n

Operator & applies AND bitwise to two integers

Result of 0x69 & 0x55

(0 1 1 0 1 0 0 1) ₂
& (0 1 0 1 0 1 0 1) ₂
(0 1 0 0 0 0 0 1) ₂

Example use of &

- & is often used to mask off bits

b is any bit → $b \& 0 =$?
 $b \& 1 =$?

```
int clear_msb(int x) {  
    return _____;  
}
```

My activities



Activities

Name

 Untitled folder

 What is the decimal value of bit pattern 10000110?

Activities



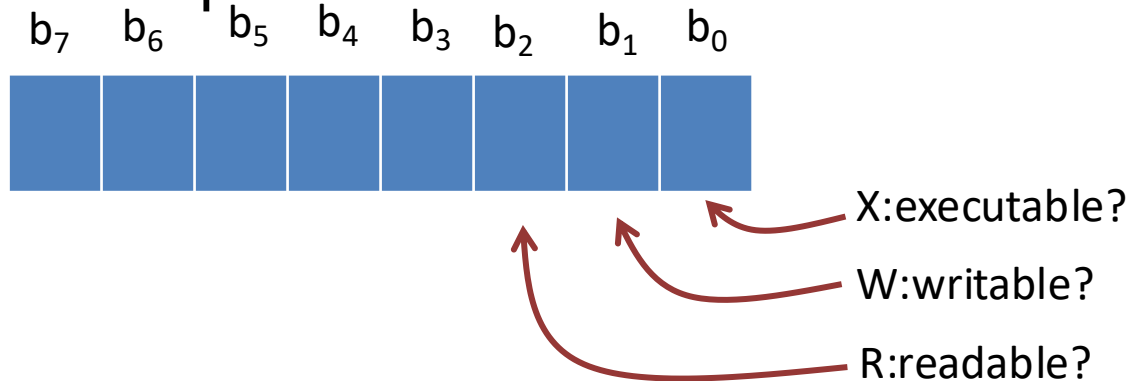
My activities

Shared activities



Example use of &

- Suppose a file's permission is encoded in 3 bits



```
void check_write_permission(char perm) {  
    if (perm & 0x02) {  
        printf("Has write permission\n");  
    } else {  
        printf("No write permission\n");  
    }  
}
```

```
char clear_write_permission(char perm) {  
    return perm & ~0x02  
}
```

Bitwise OR: |

x	y	x OR y
0	0	0
0	1	1
1	0	1
1	1	1

Operator | applies OR bitwise to two integers

$$\begin{array}{r} \text{Result of } 0x69 \mid 0x55 \\ (01101001)_2 \\ | (01010101)_2 \\ \hline (01111101)_2 \end{array}$$

Example use of |

- | can be used to turn some bits on

$b \mid 1 =$?

$b \mid 0 =$?

```
char set_write_permission(char x) {  
    return x | 0x02;  
}
```

Bitwise XOR: ^

x	y	x XOR y
0	0	0
0	1	1
1	0	1
1	1	0

Operator ^ applies XOR bitwise to two integers

$$\begin{array}{r} \text{result of } 0x69 \wedge 0x55 \\ \begin{array}{r} (0\ 1\ 1\ 0\ 1\ 0\ 0\ 1)_2 \\ \wedge (0\ 1\ 0\ 1\ 0\ 1\ 0\ 1)_2 \\ \hline (0\ 0\ 1\ 1\ 1\ 1\ 0\ 0)_2 \end{array} \end{array}$$

Example use of XOR

- $\wedge$ can be used to selectively toggle a bit

$$\begin{aligned} b \wedge 1 &= \text{red square} \\ b \wedge 0 &= \text{red square} \end{aligned}$$

```
char toggle_write_permission(char x) {  
    return x ^ 0x02;  
}
```

Bitwise left-shift: <<

$x \ll y$, treat x as a bit-vector, shift x left by y positions

- Throw away bits shifted out on the left
- Fill in 0's on the right

result of $0x69 \ll 3$? $(0\ 1\ 1\ 0\ 1\ 0\ 0\ 1)_2$

$= (0\ 1\ 0\ 0\ 1\ 0\ 0\ 0)_2$

Bitwise right-shift: >>

- $x \gg y$, shift bit-vector x right by y positions
 - Throw away bits shifted out on the right
 - **Logical shift**: Fill with 0's on left
 - **Arithmetic shift**: Replicate msb on the left

Result of (logical) right-shift: $0xa9 \gg 3$ $(1\ 0\ 1\ 0\ 1\ 0\ 0\ 1)_2$

$= (0\ 0\ 0\ 1\ 0\ 1\ 0\ 1)_2$

Result of (arithmetic) right-shift: $0xa9 \gg 3$ $= (1\ 1\ 1\ 1\ 0\ 1\ 0\ 1)_2$

Which shift is used in C ?

Arithmetic shift for signed numbers

Logical shifting on unsigned numbers

```
int a = 1;  
a = a>>31;
```

```
int b = -1;  
b = b >>31;
```

```
a= ??  b= ??
```

```
unsigned int b = 129;  
b = b >> 4;
```

```
b = 8
```

Example use of shift

```
int multiply_by_powers_of_two(int x, int p)
{
    return x << p;
}
```

```
int divide_by_powers_of_two(int x, int p)
{
    return x >> p;
}
```

Caveat: right-shift rounds down, different from integer division “/” which rounds towards zero.
e.g. $-1 \gg 1 = -1$, but $-1/2 = 0$

Example use of shift

```
// clear bit at position pos
// rightmost bit is at 0th pos

int clear_bit_at_pos(int x, int pos)
{
    unsigned int mask = ~(1 << pos);
    return x & mask;
}
```

Lesson plan

- Bitwise operators
- Control flow
- Pointers

C's Control flow

- Same as Java
- conditional:
 - if ... else if... else
 - switch
- loops: while, for
 - continue
 - break

goto statements allow jump anywhere

`goto label`

```
while (cond) {  
    ...  
}  
B:  
...
```



Any control flow primitive can be expressed as a bunch of goto's.

```
A:  
if (cond = false) goto B;  
...  
goto A  
B:  
...
```

```
for(...) {  
    for(...) {  
        for(...) {  
            goto error  
        }  
    }  
}
```

```
error:  
    code handling error
```

The only acceptable scenario for using goto

Avoid goto's whenever possible

Edgar Dijkstra: Go To Statement Considered Harmful

Go To Statement Considered Harmful

Key Words and Phrases: go to statement, jump instruction, branch instruction, conditional clause, alternative clause, repetitive clause, program intelligibility, program sequencing

CR Categories: 4.22, 5.23, 5.24

EDITOR:

For a number of years I have been familiar with the observation that the quality of programmers is a decreasing function of the density of **go to** statements in the programs they produce. More recently I discovered why the use of the **go to** statement has such disastrous effects, and I became convinced that the **go to** statement should be abolished from all "higher level" programming languages (i.e. everything except, perhaps, plain machine code). At that time I did not attach too much importance to this discovery; I now submit my considerations for publication because in very recent discussions in which the subject turned up, I have been urged to do so.

My first remark is that, although the programmer's activity ends when he has constructed a correct program, the process taking place under control of his program is the true subject matter of his activity, for it is this process that has to accomplish the desired effect; it is this process that in its dynamic behavior has to satisfy the desired specifications. Yet, once the program has been made, the "making" of the corresponding process is dele-

dynamic progress is only characterized when we also give to which call of the procedure we refer. With the inclusion of procedures we can characterize the progress of the process via a sequence of textual indices, the length of this sequence being equal to the dynamic depth of procedure calling.

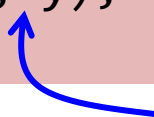
Let us now consider repetition clauses (like, **while B repeat A** or **repeat A until B**). Logically speaking, such clauses are now superfluous, because we can express repetition with the aid of recursive procedures. For reasons of realism I don't wish to exclude them: on the one hand, repetition clauses can be implemented quite comfortably with present day finite equipment; on the other hand, the reasoning pattern known as "induction" makes us well equipped to retain our intellectual grasp on the processes generated by repetition clauses. With the inclusion of the repetition clauses textual indices are no longer sufficient to describe the dynamic progress of the process. With each entry into a repetition clause, however, we can associate a so-called "dynamic index," inexorably counting the ordinal number of the corresponding current repetition. As repetition clauses (just as procedure calls) may be applied nestedly, we find that now the progress of the process can always be uniquely characterized by a (mixed) sequence of textual and/or dynamic indices.

The main point is that the values of these indices are outside programmer's control; they are generated (either by the write-up of his program or by the dynamic evolution of the process) whether

C scope rules

- Local variable: defined within a procedure
 - Scope: within the procedure
- Global variable: defined outside of any procedure
 - Scope: within the file unless exported via *.h

```
int x = 0;
int y = 1;
void swap(int x, int y) {
    int tmp = x;
    x = y;
    y = x;
}
int main() {
    swap(x, y);
}
```



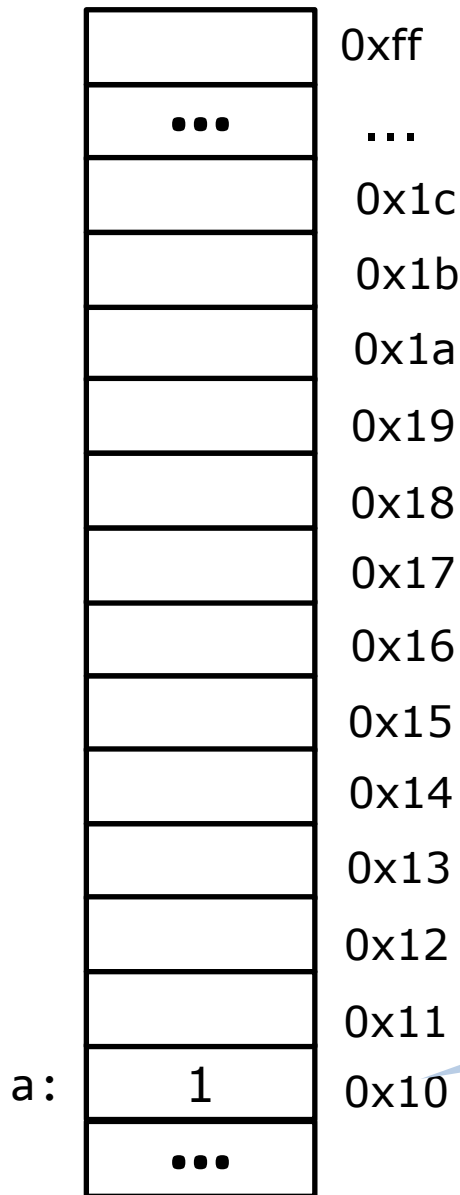
Function arguments are passed-by-value

Pointers

Pointer is a memory address

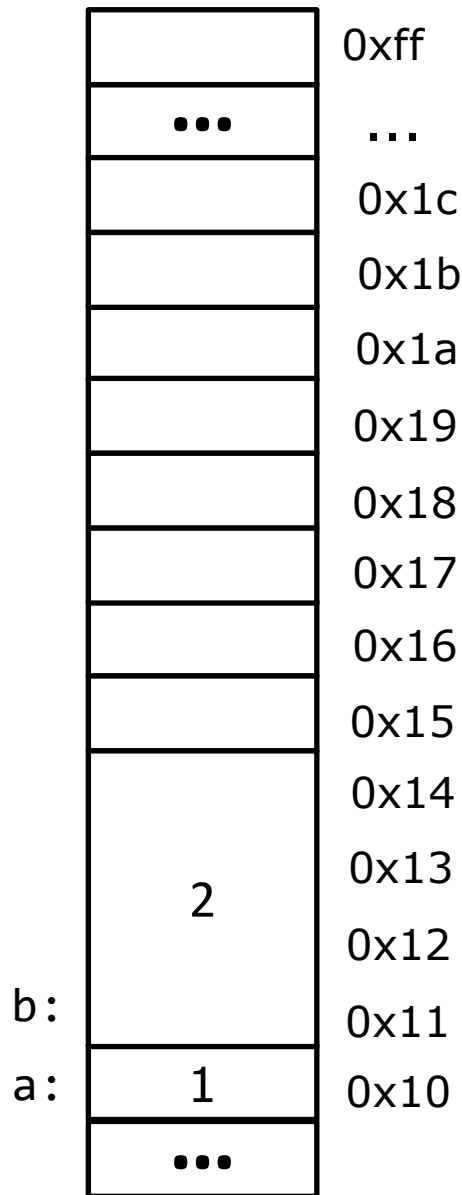
Pointer

```
char a = 1;
```



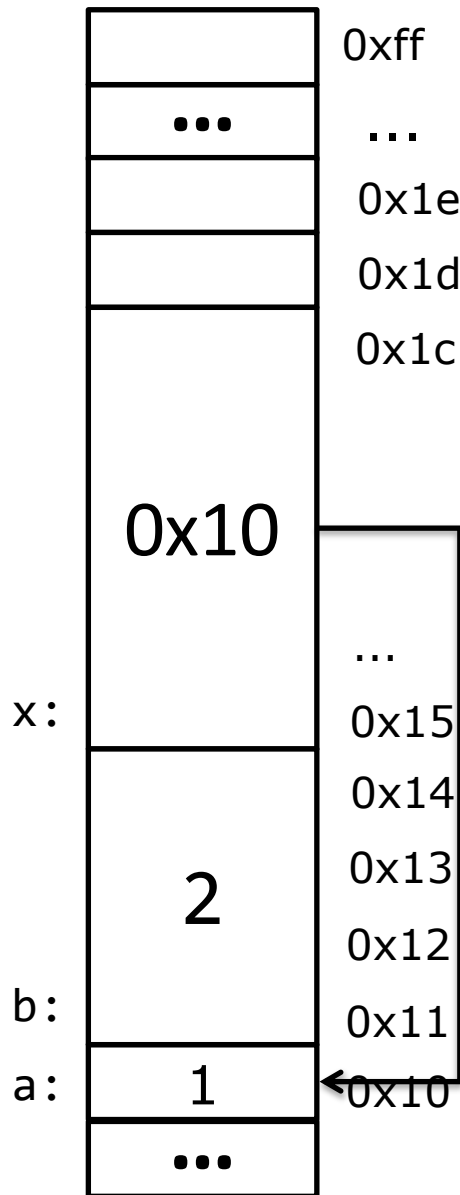
Addresses are 8-byte long on 64-bit machine; for brevity, I omit leading 0s.

Pointer



```
char a = 1;  
int b = 2;
```

Pointer



```
char a = 1;
```

```
int b = 2;
```

```
char *x;
```

```
x = &a;
```

Same as: `char* x;`
You pronounce typename
from right to left

`&` gives address
of variable

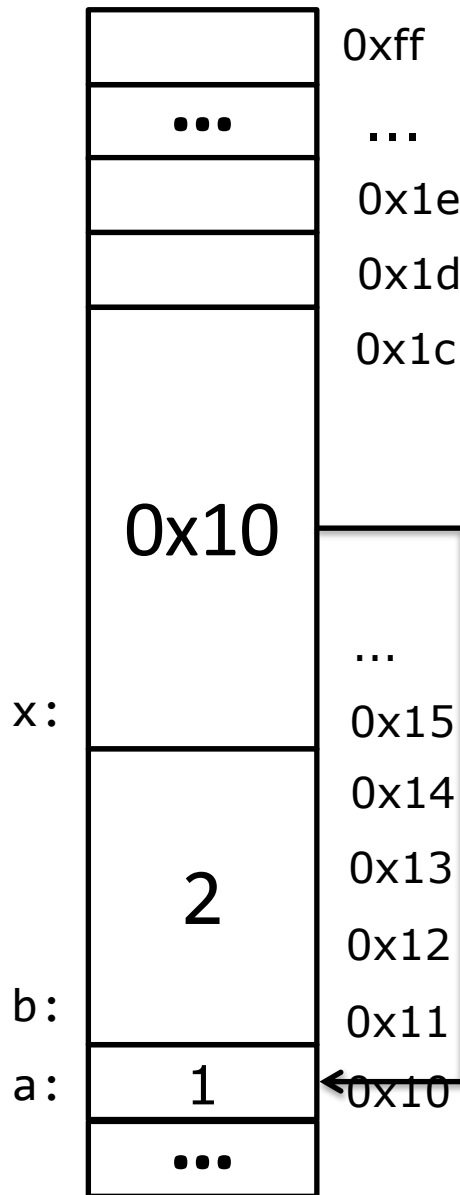
Can be combined as:
`char *x = &a;`

what happens if I write
`char x = &a;`

type mismatch!

Pointer

```
char a = 1;  
int b = 2;  
char *x = &a;  
printf("x=%p\n", x);
```



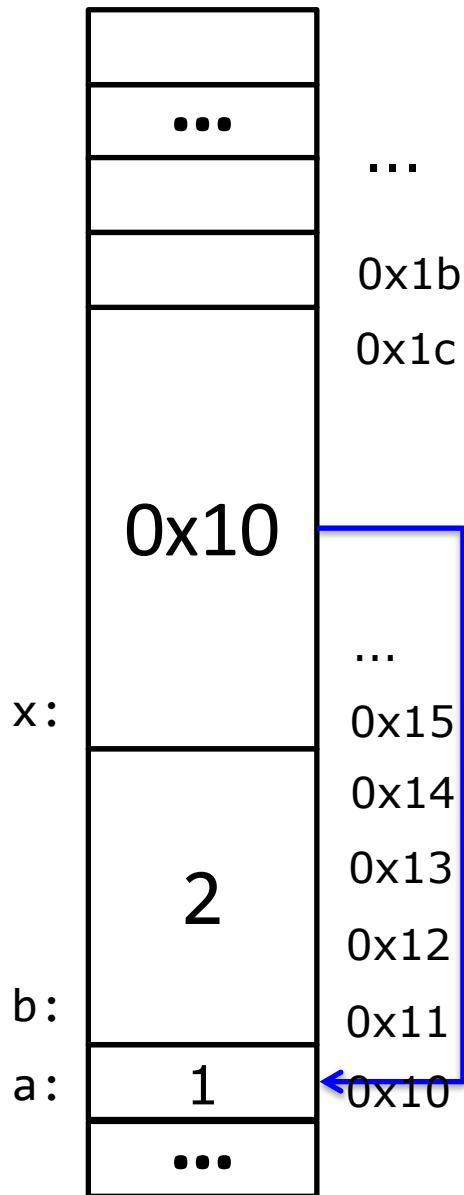
You can print the value of a pointer variable with %p (leading zeros are not printed)
0x10



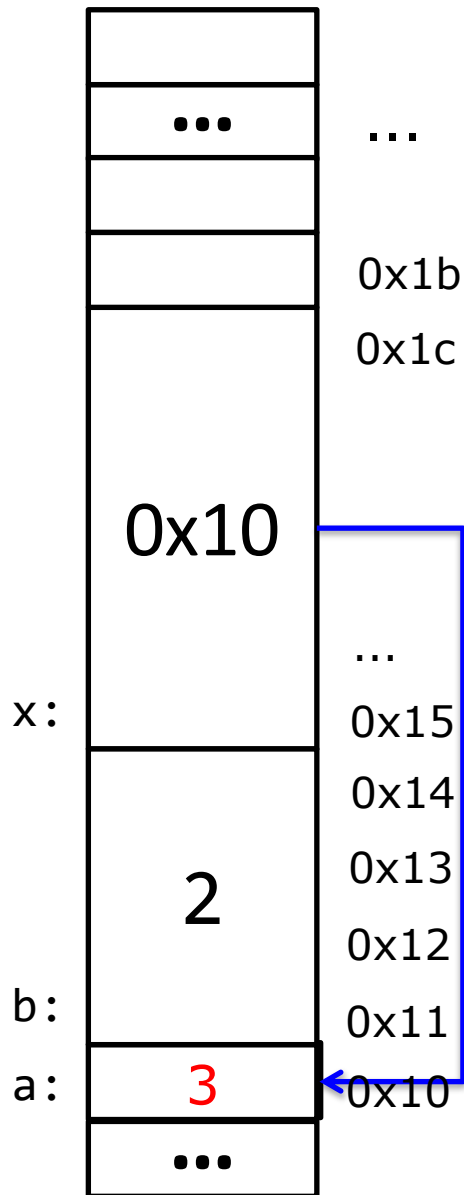
Size of pointer on a 64-bit machine?
8 bytes

Pointer

```
char a = 1;  
int b = 2;  
char *x = &a;
```



Pointer



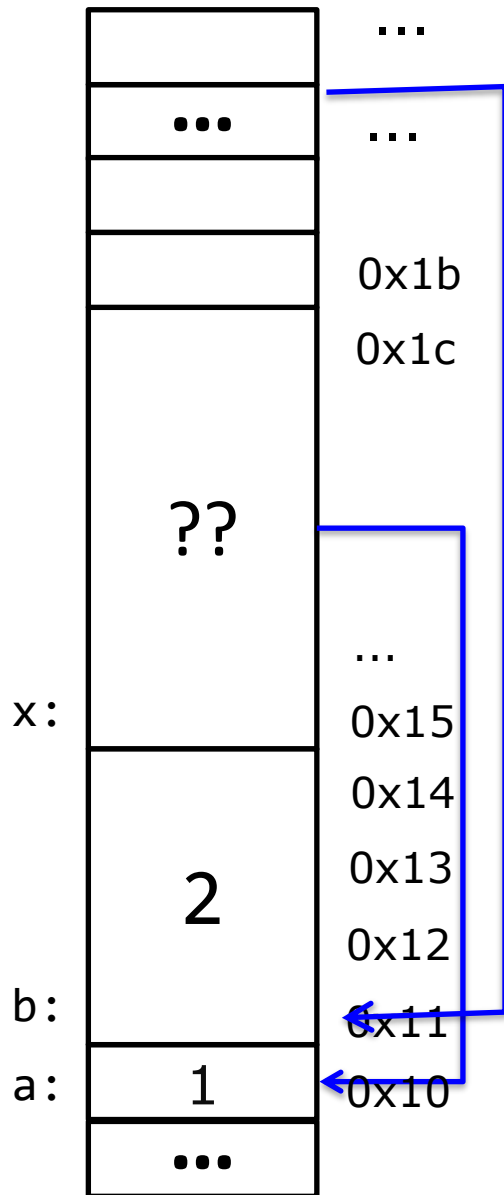
```
char a = 1;  
int b = 2;  
char *x = &a;
```

```
*x = 3;
```

* operator dereferences a pointer, not to be confused with the * in (char *) which is part of typename

Value of variable a after this statement?

Pointer



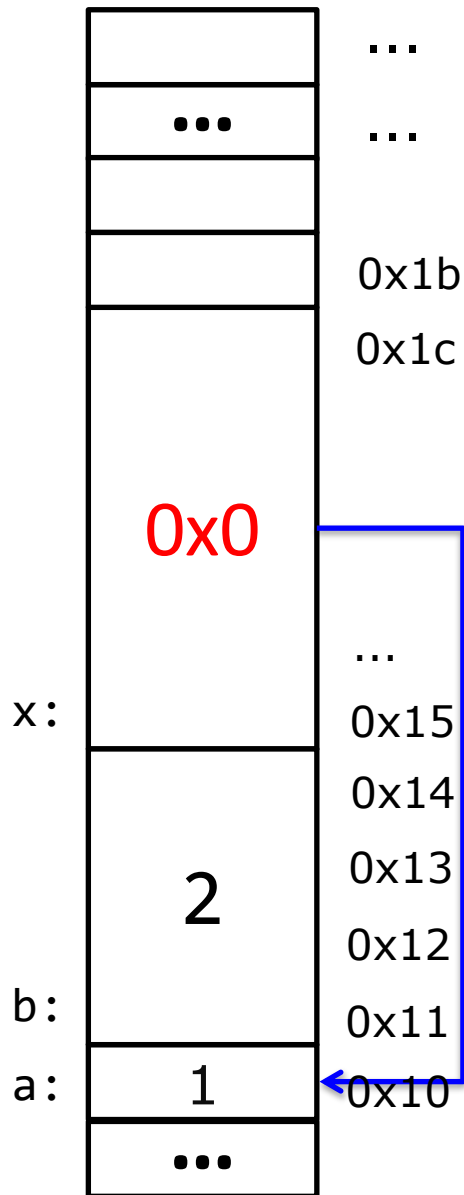
```
char a = 1;  
int b = 2;  
char *x = &a;
```

what if x is uninitialized?

```
*x = 3;
```

Dereferencing an arbitrary address value may result in "Segmentation fault" or a random memory write

Pointer



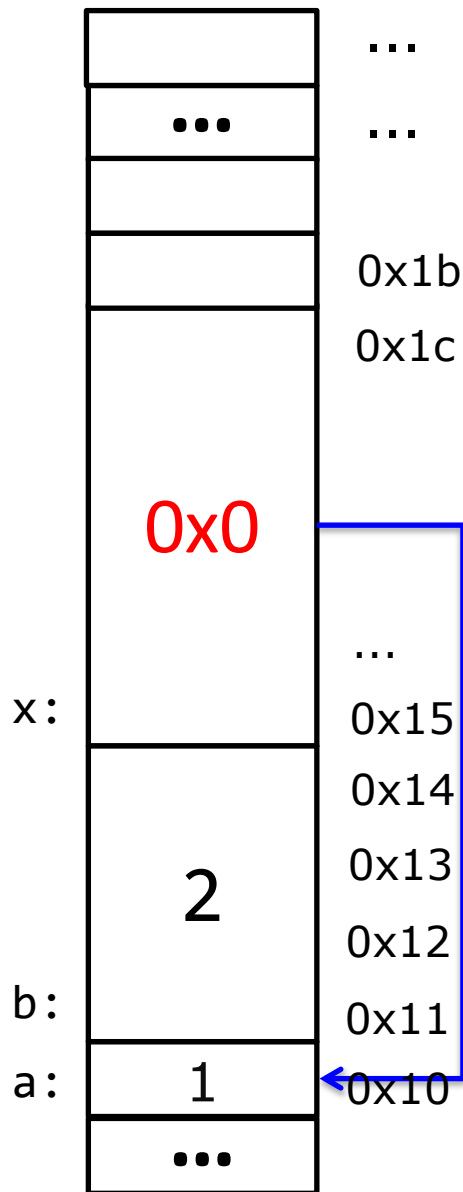
```
char a = 1;  
int b = 2;  
char *x = NULL;
```

Always initialize
pointers!

```
*x = 3;
```

Dereferencing NULL pointer
definitely results in
“Segmentation fault”

Pointer

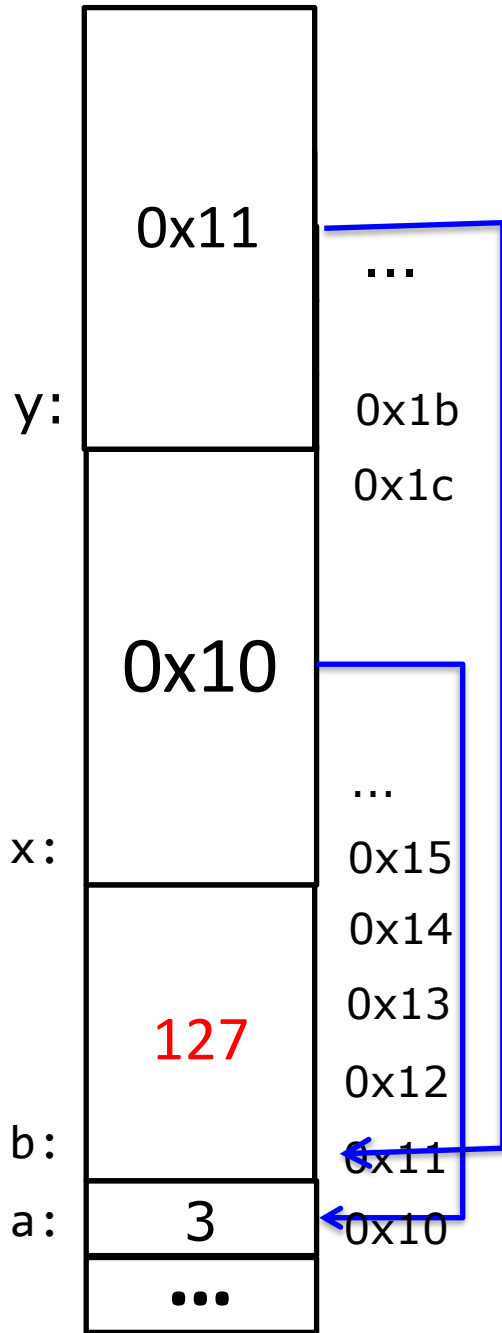


```
char a = 1;  
int b = 2;  
char *x = NULL;
```

```
*x = 3;
```

```
(gdb) r  
Starting program: /oldhome/jinyang/a.out  
  
Program received signal SIGSEGV, Segmentation fault.  
0x00000000004005ef in main () at foo.c:16  
16          *x = 3;  
(gdb) p x  
$1 = 0x0  
(gdb)
```

Pointer has different types



```
char a = 1;  
int b = 2;  
char *x = &a;  
  
*x = 3;
```

```
int *y = &b;  
  
*y = 127;
```

what if I write
char *y = &b;

My activities



Activities

Name

 Untitled folder

 What is the decimal value of bit pattern 10000110?

Activities

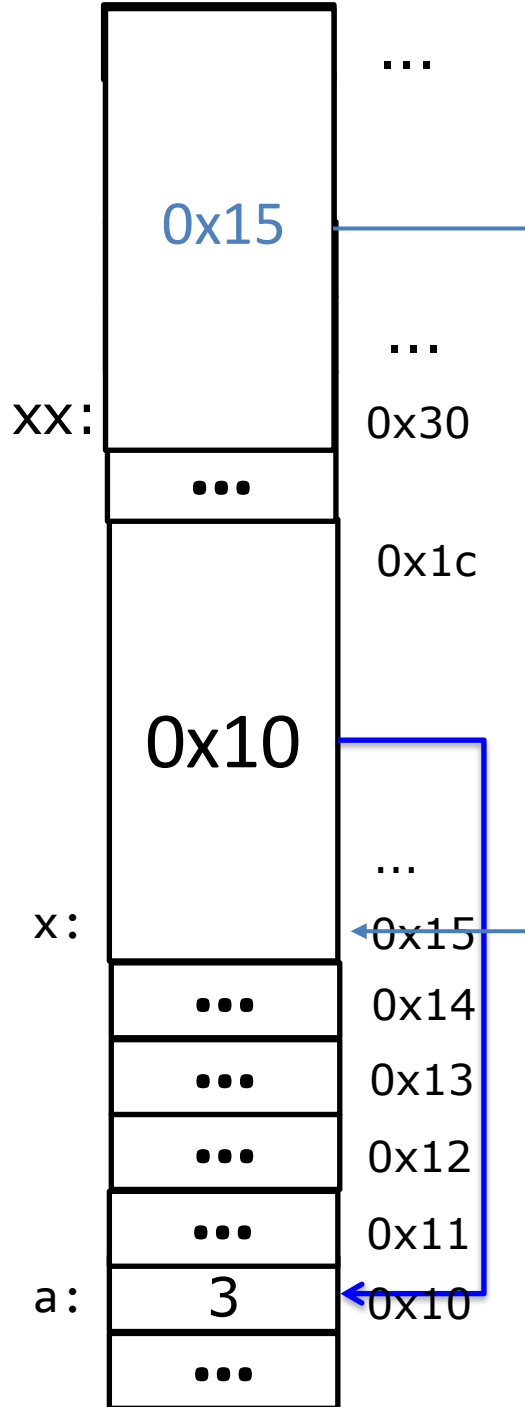


My activities

Shared activities



Double Pointer



```
char a = 1;
int b = 2;
char *x = &a;
*x = 3;
```

```
char **xx = &x;
```

char **xx is the
same as char** xx;

Same as:
char **xx;
xx = &x;

what if I write
char *xx = &x;

```
printf("xx=%p *xx=%p **xx=%d\n", xx, *xx, **xx);
```

Common confusions on *

* has two meanings!!

1. part of a pointer type name, e.g. `char *`, `char **`, `int *`
2. the deference operator.

```
char a = 1;  
char *p = &a;  
*p = 2;
```

```
char *b, *c;  
char **d, **e;
```

```
char *f=p, *g=p;  
char **m=&p, **n=&p;
```

C's syntax for declaring multiple pointer variables on one line

`char* b, c;` does not work

C's syntax for declaring and initializing multiple pointer variables on one line

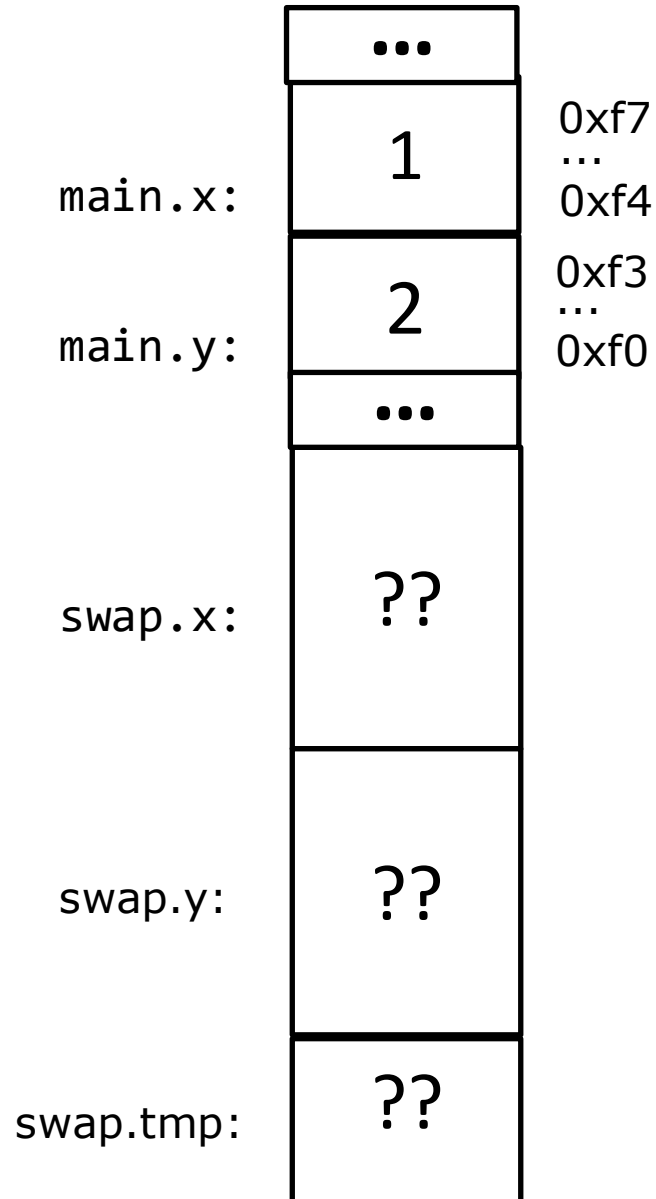
Pass pointers to function

```
void swap(int* x, int* y)
{
    int tmp = *x;
    *x = *y;
    *y = tmp;
}

int main()
{
    int x = 1;
    int y = 2;
    swap(&x, &y);

    printf("x:%d, y:%d", x, y);
}
```

Size and value of x, y, tmp
in swap upon function entrance?

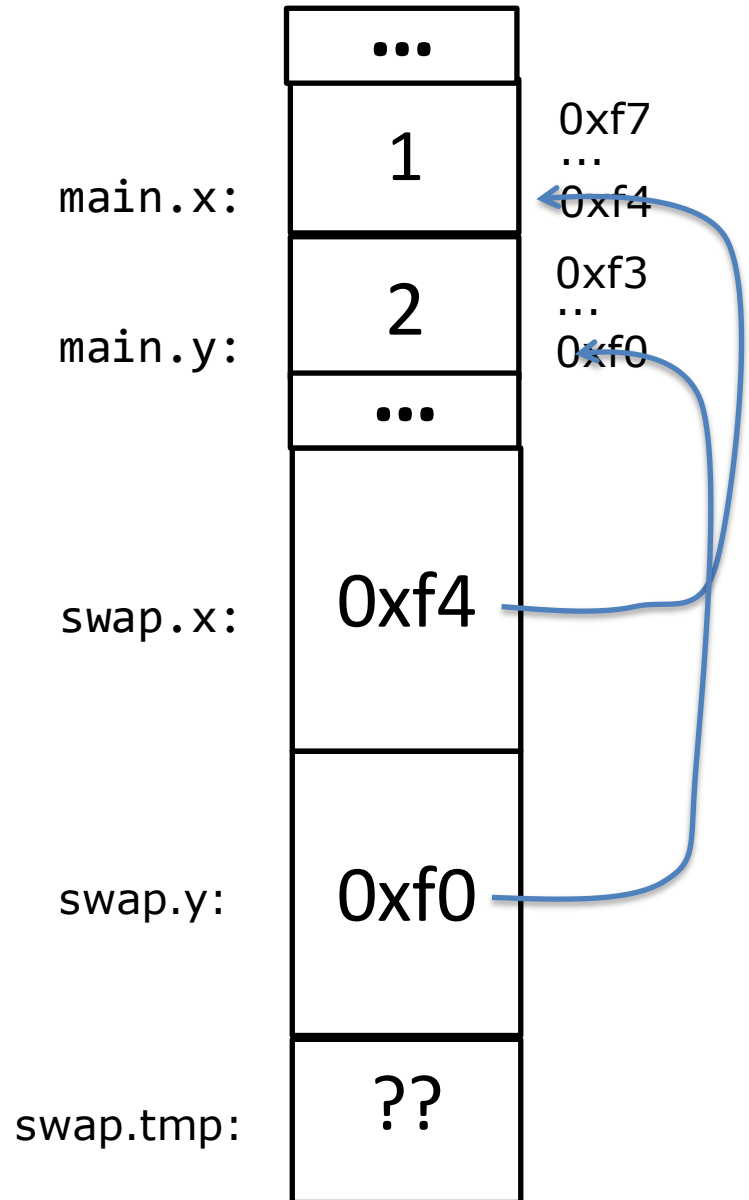


Pass pointers to function

```
void swap(int* x, int* y)
{
    int tmp = *x;
    *x = *y;
    *y = tmp;
}

int main()
{
    int x = 1;
    int y = 2;
    swap(&x, &y);

    printf("x:%d, y:%d", x, y);
}
```

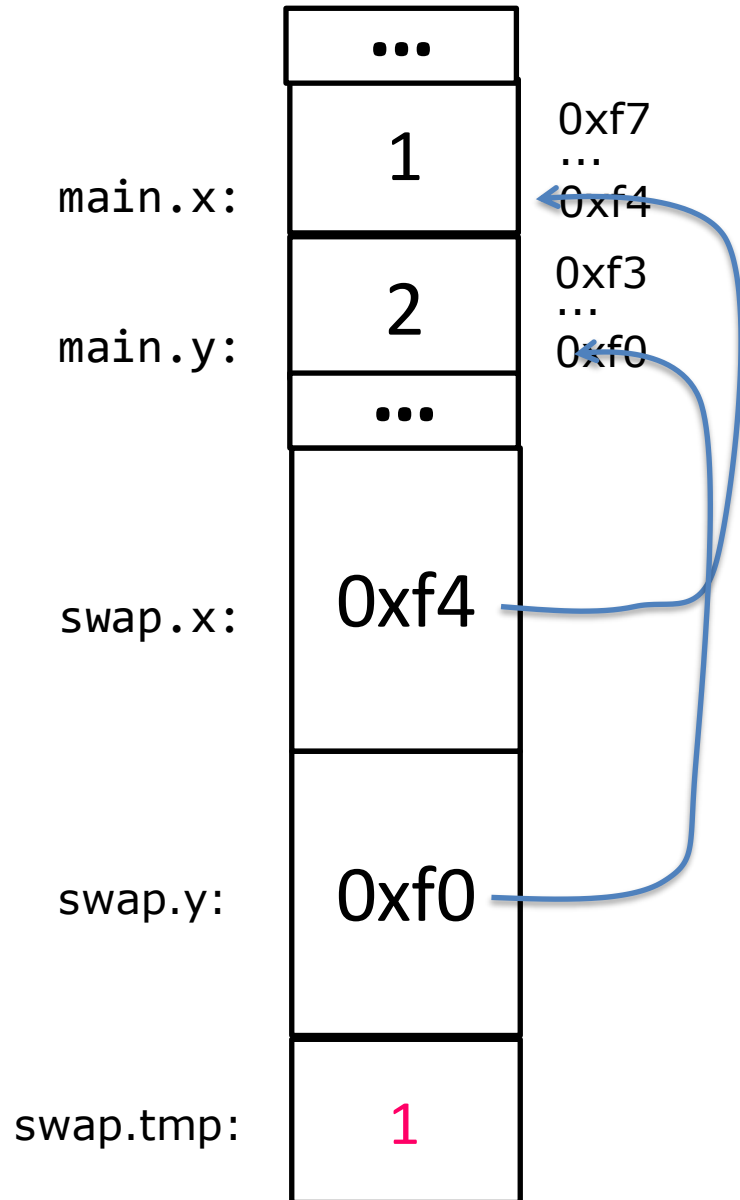


Pass pointers to function

```
void swap(int* x, int* y)
{
    int tmp = *x;
    *x = *y;
    *y = tmp;
}

int main()
{
    int x = 1;
    int y = 2;
    swap(&x, &y);

    printf("x:%d, y:%d", x, y);
}
```

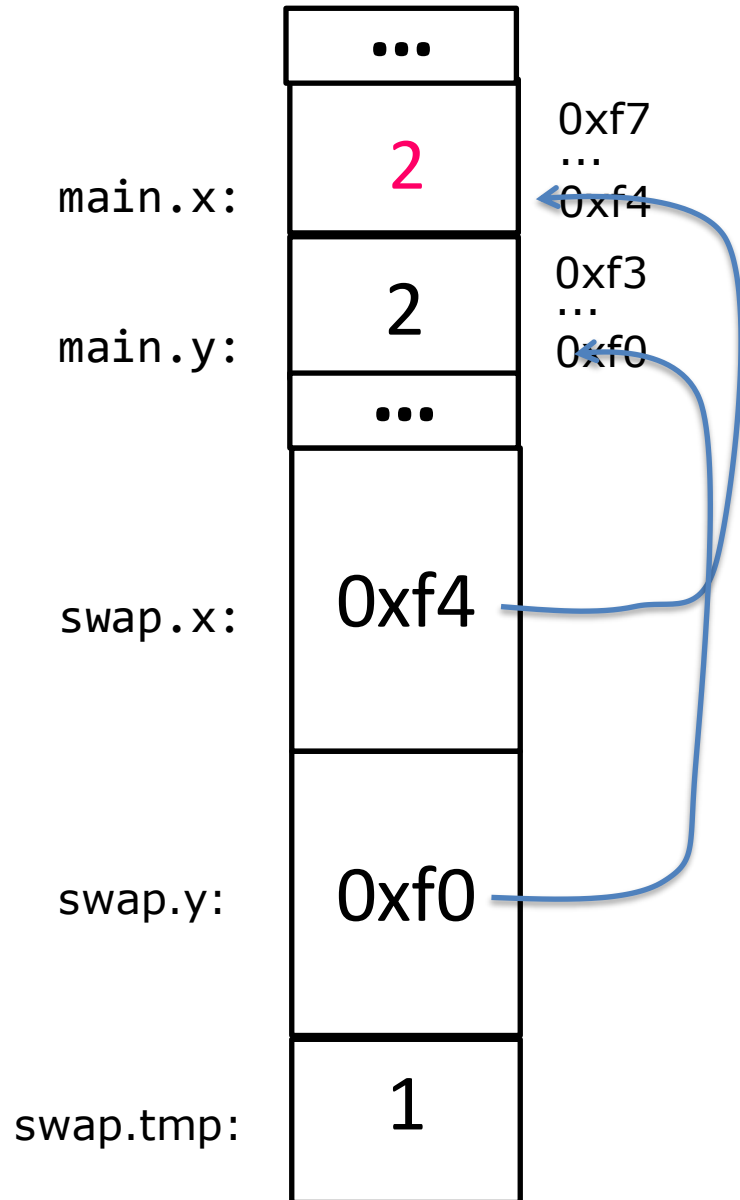


Pass pointers to function

```
void swap(int* x, int* y)
{
    int tmp = *x;
    *x = *y;
    *y = tmp;
}

int main()
{
    int x = 1;
    int y = 2;
    swap(&x, &y);

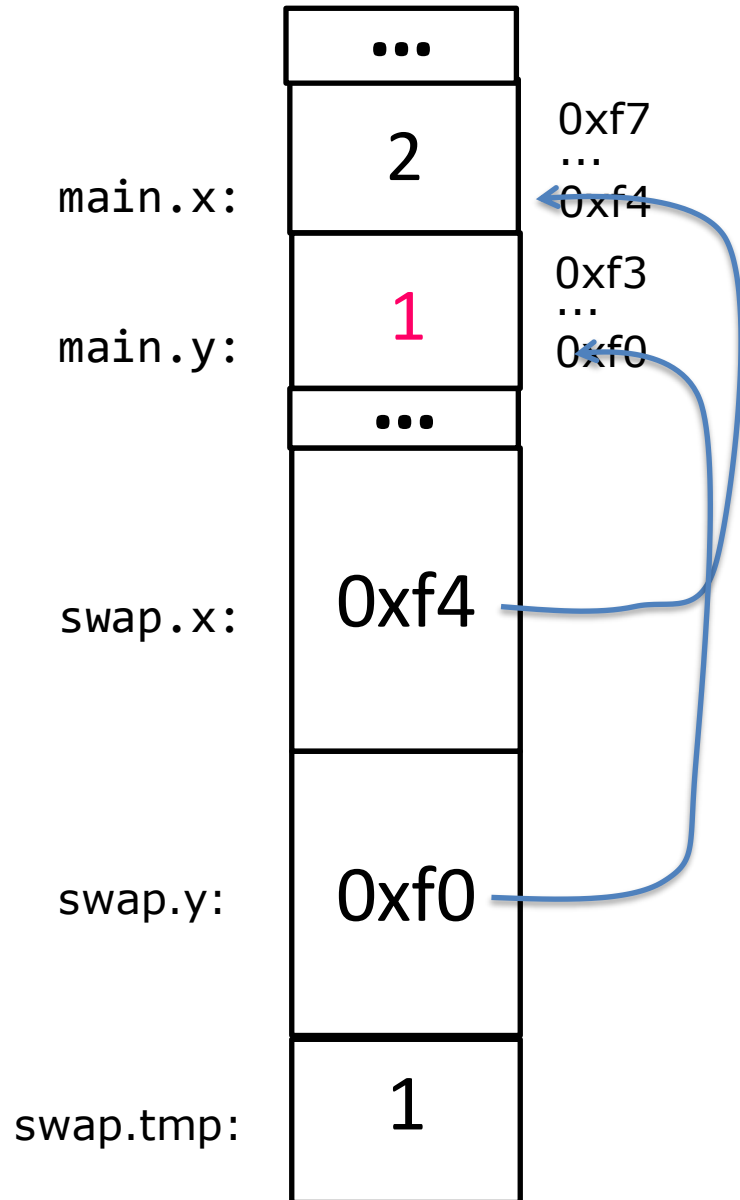
    printf("x:%d, y:%d", x, y);
}
```



Pass pointers to function

```
void swap(int* x, int* y)
{
    int tmp = *x;
    *x = *y;
    *y = tmp;
}
→
int main()
{
    int x = 1;
    int y = 2;
    swap(&x, &y);

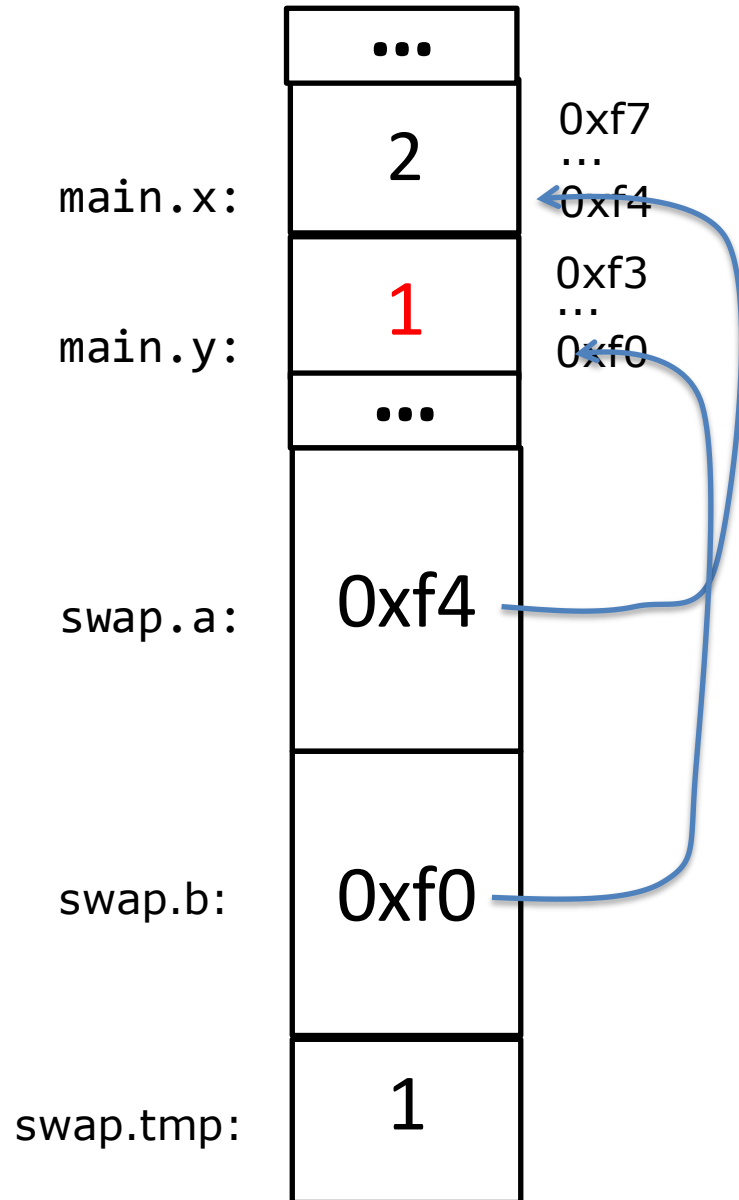
    printf("x:%d, y:%d", x, y);
}
```



Pass pointers to function

```
void swap(int* a, int* b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}

int main()
{
    int x = 1;
    int y = 2;
    swap(&x, &y);
    printf("x:%d, y:%d", x, y);
}
```



Summary

- Bitwise Operators: `&`, `|`, `^`, `>>`, `<<`
- Control flow and variable scopes
- Pointers
 - Pointers are memory addresses: `p = &x`; (p has addr of var x)
 - `*p` ... (read/write the variable pointed to by p)
- Pass pointers to functions
 - Allows a callee to change variables defined in caller